

## Q1: Implement a Traffic Light using 3 LEDs and Arduino System. It Should Mimic Real-World Conditions.

### Required Components:

- Arduino Uno
  - Red LED (Stop)
  - Yellow LED (Ready)
  - Green LED (Go)
  - 220Ω Resistors (3x)
  - LCD Display (16x2) with I2C module
  - Jumper Wires
  - Breadboard
- 

### Circuit Connections:

#### 1. LED Connections

- Connect the **Red LED** (Stop) to **Pin 3** via a **220Ω resistor**.
- Connect the **Yellow LED** (Ready) to **Pin 4** via a **220Ω resistor**.
- Connect the **Green LED** (Go) to **Pin 5** via a **220Ω resistor**.
- The **negative terminals** of all LEDs should be connected to **GND**.

#### 2. LCD Connections

- **VCC** → **5V**
  - **GND** → **GND**
  - **SDA** → **A4** (Arduino I2C SDA)
  - **SCL** → **A5** (Arduino I2C SCL)
- 

### Full Working Code:

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#define RED_LED 3
```

```
#define YELLOW_LED 4
```

```
#define GREEN_LED 5
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // Adjust the I2C address if needed
```

```
void setup() {
```

```
    pinMode(REDA_LED, OUTPUT);
```

```
    pinMode(YELLOW_LED, OUTPUT);
```

```
    pinMode(GREEN_LED, OUTPUT);
```

```
    lcd.begin(16, 2);
```

```
    lcd.backlight();
```

```
}
```

```
void loop() {
```

```
    // Red Light (Stop)
```

```
    digitalWrite(REDA_LED, HIGH);
```

```
    digitalWrite(YELLOW_LED, LOW);
```

```
    digitalWrite(GREEN_LED, LOW);
```

```
    lcd.clear();
```

```
    lcd.setCursor(0, 0);
```

```
    lcd.print("STOP");
```

```
    delay(5000); // Wait 5 seconds
```

```
    // Yellow Light (Ready)
```

```
    digitalWrite(REDA_LED, LOW);
```

```
    digitalWrite(YELLOW_LED, HIGH);
```

```
    digitalWrite(GREEN_LED, LOW);
```

```
    lcd.clear();
```

```
    lcd.setCursor(0, 0);
```

```
    lcd.print("READY");
```

```
delay(2000); // Wait 2 seconds

// Green Light (Go)
digitalWrite(RED_LED, LOW);
digitalWrite(YELLOW_LED, LOW);
digitalWrite(GREEN_LED, HIGH);
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("GO");
delay(5000); // Wait 5 seconds
}
```

---

This code mimics real-world traffic light behavior while displaying the current signal status on the LCD. Now, let me know if you want any modifications, or I can proceed to **Q2**.

## **Q2: Use LDR Sensor and LEDs. Design the Circuit to Glow the LED at Various Different Values with Serial Monitor and LCD Display.**

### **Required Components:**

- Arduino Uno
  - LDR (Light Dependent Resistor)
  - 10k $\Omega$  Resistor
  - LED (Any color)
  - 220 $\Omega$  Resistor
  - LCD Display (16x2) with I2C module
  - Jumper Wires
  - Breadboard
- 

### **Circuit Connections:**

#### **1. LDR Sensor Setup:**

- One leg of the **LDR** to **5V**
- The other leg to **A0** (Analog Input)

- **10k $\Omega$  Resistor** between **A0** and **GND** (Pull-down resistor)

## 2. LED Setup:

- Connect the **LED** to **Pin 3** with a **220 $\Omega$  resistor**
- The **negative terminal** of the LED goes to **GND**

## 3. LCD Connections:

- **VCC  $\rightarrow$  5V**
- **GND  $\rightarrow$  GND**
- **SDA  $\rightarrow$  A4**
- **SCL  $\rightarrow$  A5**

---

### Full Working Code:

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#define LDR_PIN A0
```

```
#define LED_PIN 3
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // Adjust address if needed
```

```
void setup() {
```

```
    pinMode(LED_PIN, OUTPUT);
```

```
    lcd.begin(16, 2);
```

```
    lcd.backlight();
```

```
    Serial.begin(9600);
```

```
}
```

```
void loop() {
```

```
    int ldrValue = analogRead(LDR_PIN); // Read LDR value
```

```
    Serial.print("LDR Value: ");
```

```
Serial.println(ldrValue);

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Light Level: ");
lcd.setCursor(0, 1);
lcd.print(ldrValue);

// LED control based on LDR value
if (ldrValue < 300) {
    digitalWrite(LED_PIN, HIGH);
} else {
    digitalWrite(LED_PIN, LOW);
}

delay(500);
}
```

---

This program reads the **LDR sensor** and displays its values on the **LCD** and **Serial Monitor**. The LED lights up in low-light conditions. Let me know if you want modifications, or I can proceed to **Q3**.

**Q3: Use Ultrasonic Sensor and 3 LEDs. Design the Circuit to Glow the LEDs at Three Different Distances with Its Display on Serial Monitor and LCD.**

**Required Components:**

- Arduino Uno
  - Ultrasonic Sensor **HC-SR04**
  - 3 LEDs (Red, Yellow, Green)
  - 3 Resistors (220Ω each)
  - LCD Display (16x2) with I2C module
  - Jumper Wires
  - Breadboard
- 

**Circuit Connections:**

**1. Ultrasonic Sensor (HC-SR04) Setup:**

- **VCC → 5V**
- **GND → GND**
- **TRIG → Pin 9**
- **ECHO → Pin 10**

**2. LED Setup:**

- **Red LED → Pin 3** (With 220Ω resistor to GND)
- **Yellow LED → Pin 4** (With 220Ω resistor to GND)
- **Green LED → Pin 5** (With 220Ω resistor to GND)

**3. LCD Connections:**

- **VCC → 5V**
  - **GND → GND**
  - **SDA → A4**
  - **SCL → A5**
- 

**Full Working Code:**

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#define TRIG 9
```

```
#define ECHO 10
```

```
#define LED_RED 3
```

```
#define LED_YELLOW 4
```

```
#define LED_GREEN 5
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // Adjust address if needed
```

```
void setup() {
```

```
    pinMode(TRIG, OUTPUT);
```

```
    pinMode(ECHO, INPUT);
```

```
    pinMode(LED_RED, OUTPUT);
```

```
    pinMode(LED_YELLOW, OUTPUT);
```

```
    pinMode(LED_GREEN, OUTPUT);
```

```
    lcd.begin(16, 2);
```

```
    lcd.backlight();
```

```
    Serial.begin(9600);
```

```
}
```

```
long getDistance() {
```

```
    digitalWrite(TRIG, LOW);
```

```
    delayMicroseconds(2);
```

```
    digitalWrite(TRIG, HIGH);
```

```
    delayMicroseconds(10);
```

```
    digitalWrite(TRIG, LOW);
```

```
    return pulseIn(ECHO, HIGH) * 0.034 / 2; // Convert time to distance in cm
```

```
}
```

```
void loop() {  
    long distance = getDistance();  
  
    Serial.print("Distance: ");  
    Serial.print(distance);  
    Serial.println(" cm");  
  
    lcd.clear();  
    lcd.setCursor(0, 0);  
    lcd.print("Distance: ");  
    lcd.print(distance);  
    lcd.print(" cm");  
  
    // LED Control  
    if (distance < 10) {  
        digitalWrite(LED_RED, HIGH);  
        digitalWrite(LED_YELLOW, LOW);  
        digitalWrite(LED_GREEN, LOW);  
        lcd.setCursor(0, 1);  
        lcd.print("Too Close!");  
    }  
    else if (distance >= 10 && distance < 20) {  
        digitalWrite(LED_RED, LOW);  
        digitalWrite(LED_YELLOW, HIGH);  
        digitalWrite(LED_GREEN, LOW);  
        lcd.setCursor(0, 1);  
        lcd.print("Medium Range");  
    }  
    else {  
        digitalWrite(LED_RED, LOW);  
        digitalWrite(LED_YELLOW, LOW);  
    }  
}
```



```
digitalWrite(LED_GREEN, HIGH);  
  
lcd.setCursor(0, 1);  
  
lcd.print("Safe Distance");  
  
}  
  
  
delay(500);  
  
}
```

---

**Working:**

1. **Ultrasonic Sensor** measures the distance and displays it on the **LCD** and **Serial Monitor**.
2. **LEDs Glow Based on Distance:**
  - **Red LED** (<10cm) → Too Close
  - **Yellow LED** (10-20cm) → Medium Range
  - **Green LED** (>20cm) → Safe Distance

Let me know if you need modifications. Shall I proceed to **Q4**?

**Q4: Design the Circuit for Servo Motor with Potentiometer. It is Expected That Servo Motor Moves with Change in Potentiometer Position. Display the Digital Values in the Serial Monitor and LCD.**

**Required Components:**

- Arduino Uno
  - **Servo Motor**
  - **Potentiometer (10k $\Omega$ )**
  - **LCD Display (16x2) with I2C module**
  - Jumper Wires
  - Breadboard
- 

**Circuit Connections:**

1. **Potentiometer Setup:**
    - **VCC** → 5V
    - **GND** → GND
    - **Middle Pin** → A0 (Analog Input Pin)
  2. **Servo Motor Setup:**
    - **VCC (Red Wire)** → 5V
    - **GND (Black/Brown Wire)** → GND
    - **Signal (Yellow/Orange Wire)** → Pin 9
  3. **LCD Connections:**
    - **VCC** → 5V
    - **GND** → GND
    - **SDA** → A4
    - **SCL** → A5
- 

**Full Working Code:**

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#include <Servo.h>
```

```
#define POT_PIN A0

#define SERVO_PIN 9


LiquidCrystal_I2C lcd(0x27, 16, 2); // Adjust address if needed

Servo myServo;


void setup() {
  pinMode(POT_PIN, INPUT);
  myServo.attach(SERVO_PIN);


  lcd.begin(16, 2);
  lcd.backlight();
  Serial.begin(9600);
}


void loop() {
  int potValue = analogRead(POT_PIN); // Read potentiometer value (0-1023)
  int servoAngle = map(potValue, 0, 1023, 0, 180); // Convert to servo angle (0-180)


  myServo.write(servoAngle); // Move servo
  Serial.print("Pot Value: ");
  Serial.print(potValue);
  Serial.print(" | Servo Angle: ");
  Serial.println(servoAngle);


  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Pot: ");
  lcd.print(potValue);
```

```
lcd.setCursor(0, 1);  
lcd.print("Angle: ");  
lcd.print(servoAngle);  
lcd.print(" deg");  
  
delay(200);  
}
```

---

#### **Working:**

1. The **potentiometer** controls the **servo motor** position.
2. The **potentiometer's analog value (0-1023)** is converted to **servo angle (0-180°)**.
3. **LCD & Serial Monitor Display:**
  - **Potentiometer Value**
  - **Servo Angle in Degrees**

Let me know if you need modifications. Shall I proceed to **Q5**?

**Q5: Design the Circuit for an Infrared Sensor. Use Two or Three LEDs to Glow at Different Values. Display the Value on Serial Monitor and LCD.**

---

**Required Components:**

- **Arduino Uno**
  - **IR Sensor** (Infrared Obstacle Detection Sensor)
  - **Three LEDs (Red, Yellow, Green)**
  - **Three Resistors (230Ω each for LEDs)**
  - **LCD Display (16x2) with I2C module**
  - Jumper Wires
  - Breadboard
- 

**Circuit Connections:**

**IR Sensor:**

- **VCC → 5V**
- **GND → GND**
- **OUT → Pin 8**

**LED Connections:**

- **Red LED (Close Detection) → Pin 3**
- **Yellow LED (Medium Distance) → Pin 4**
- **Green LED (Far / No Obstacle) → Pin 5**

**LCD Display Connections:**

- **VCC → 5V**
  - **GND → GND**
  - **SDA → A4**
  - **SCL → A5**
- 

**Full Working Code:**

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#define IR_SENSOR_PIN 8
```

```
#define LED_RED 3
```

```
#define LED_YELLOW 4
```

```
#define LED_GREEN 5
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // Adjust the I2C address if needed
```

```
void setup() {
```

```
    pinMode(IR_SENSOR_PIN, INPUT);
```

```
    pinMode(LED_RED, OUTPUT);
```

```
    pinMode(LED_YELLOW, OUTPUT);
```

```
    pinMode(LED_GREEN, OUTPUT);
```

```
    lcd.begin(16, 2);
```

```
    lcd.backlight();
```

```
    Serial.begin(9600);
```

```
}
```

```
void loop() {
```

```
    int sensorValue = digitalRead(IR_SENSOR_PIN); // Read IR sensor value
```

```
    lcd.clear();
```

```
    lcd.setCursor(0, 0);
```

```
    lcd.print("IR Value: ");
```

```
    lcd.print(sensorValue);
```

```
    Serial.print("IR Sensor Value: ");
```

```
    Serial.println(sensorValue);
```

```
if (sensorValue == 0) { // Obstacle detected

  digitalWrite(LED_RED, HIGH);

  digitalWrite(LED_YELLOW, LOW);

  digitalWrite(LED_GREEN, LOW);

  lcd.setCursor(0, 1);

  lcd.print("Obstacle Close!");

} else { // No obstacle detected

  digitalWrite(LED_RED, LOW);

  digitalWrite(LED_YELLOW, LOW);

  digitalWrite(LED_GREEN, HIGH);

  lcd.setCursor(0, 1);

  lcd.print("No Obstacle");

}

delay(500);

}
```

---

**Working:**

1. **IR Sensor detects obstacles.**
  - **If an object is detected:**
    - The **Red LED** glows
    - **"Obstacle Close!"** is displayed on the LCD.
  - **If no object is detected:**
    - The **Green LED** glows
    - **"No Obstacle"** is displayed on the LCD.
2. **The IR Sensor values are displayed on the Serial Monitor and LCD.**

---

This code will **properly control LEDs and display information on the LCD.**  
Let me know if you need modifications. Shall I proceed to **Q6**?

**Q6: Design the Circuit for a DC Motor with Relay. It Should Glow Two LEDs to Represent ON/OFF Status of the Motor and Display Status on LCD.**

---

**Required Components:**

- **Arduino Uno**
  - **DC Motor**
  - **Relay Module (5V)**
  - **Transistor (e.g., BC547)**
  - **Diode (1N4007)**
  - **Two LEDs (Red for OFF, Green for ON)**
  - **Two Resistors (230Ω for LEDs)**
  - **LCD Display (16x2) with I2C module**
  - **Jumper Wires**
  - **Breadboard**
- 

**Circuit Connections:**

**Relay Module:**

- **VCC → 5V**
- **GND → GND**
- **IN (Control Pin) → Pin 7**

**DC Motor:**

- **Connected to Relay Normally Open (NO) Terminal**
- **Relay Common (COM) Terminal → 5V**

**LEDs:**

- **Red LED (Motor OFF) → Pin 3**
- **Green LED (Motor ON) → Pin 4**

**LCD Display Connections:**

- **VCC → 5V**
- **GND → GND**
- **SDA → A4**
- **SCL → A5**



---

**Full Working Code:**

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#define RELAY_PIN 7
```

```
#define LED_RED 3
```

```
#define LED_GREEN 4
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // Adjust address if needed
```

```
void setup() {
```

```
    pinMode(RELAY_PIN, OUTPUT);
```

```
    pinMode(LED_RED, OUTPUT);
```

```
    pinMode(LED_GREEN, OUTPUT);
```

```
    lcd.begin(16, 2);
```

```
    lcd.backlight();
```

```
    Serial.begin(9600);
```

```
}
```

```
void loop() {
```

```
    // Turn Motor ON
```

```
    digitalWrite(RELAY_PIN, HIGH);
```

```
    digitalWrite(LED_GREEN, HIGH);
```

```
    digitalWrite(LED_RED, LOW);
```

```
    lcd.clear();
```

```
    lcd.setCursor(0, 0);
```

```
lcd.print("Motor Status: ON");  
Serial.println("Motor ON");  
  
delay(5000); // Keep motor ON for 5 seconds  
  
// Turn Motor OFF  
digitalWrite(RELAY_PIN, LOW);  
digitalWrite(LED_GREEN, LOW);  
digitalWrite(LED_RED, HIGH);  
  
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("Motor Status: OFF");  
Serial.println("Motor OFF");  
  
delay(5000); // Keep motor OFF for 5 seconds  
}
```

---

### Working:

1. **When the motor is ON:**
    - **Relay is activated**
    - **Green LED glows**
    - **LCD displays "Motor Status: ON"**
    - **Serial Monitor logs "Motor ON"**
  2. **When the motor is OFF:**
    - **Relay is deactivated**
    - **Red LED glows**
    - **LCD displays "Motor Status: OFF"**
    - **Serial Monitor logs "Motor OFF"**
-

This setup ensures the motor works with a relay and provides **visual and LCD feedback**.  
Let me know if you need modifications. Shall I proceed to **Q7**?

## **Q7: Design the Circuit Using Potentiometer and LCD Display to Scroll 'YOUR NAME' on LCD Screen.**

---

### **Required Components:**

- **Arduino Uno**
  - **LCD Display (16x2) with I2C Module**
  - **Potentiometer (10k $\Omega$ ) for Speed Control**
  - Jumper Wires
  - Breadboard
- 

### **Circuit Connections:**

#### **LCD Display (I2C) Connections:**

- **VCC  $\rightarrow$  5V**
- **GND  $\rightarrow$  GND**
- **SDA  $\rightarrow$  A4**
- **SCL  $\rightarrow$  A5**

#### **Potentiometer Connections:**

- **Left Pin  $\rightarrow$  5V**
  - **Middle Pin (Output)  $\rightarrow$  A0 (Analog Pin)**
  - **Right Pin  $\rightarrow$  GND**
- 

### **Full Working Code:**

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // Adjust address if needed
```

```
#define POT_PIN A0
```

```
void setup() {  
    lcd.begin(16, 2);  
    lcd.backlight();  
}  
  
void loop() {  
    int potValue = analogRead(POT_PIN); // Read potentiometer value (0-1023)  
    int scrollSpeed = map(potValue, 0, 1023, 100, 500); // Map to delay range  
  
    String name = " ABHISHEK "; // Your Name  
  
    for (int i = 0; i < name.length(); i++) {  
        lcd.clear();  
        lcd.setCursor(0, 0);  
        lcd.print(name.substring(i)); // Print substring for scrolling  
        delay(scrollSpeed); // Adjust scroll speed based on potentiometer  
    }  
}
```

---

#### **Working:**

1. **The LCD displays your name (e.g., "ABHISHEK") and scrolls it across the screen.**
  2. **The potentiometer controls the scrolling speed.**
    - **Turn the potentiometer to increase/decrease speed.**
  3. **Your name continuously scrolls, repeating after the full display.**
- 

This setup ensures **smooth scrolling of your name with speed control** via the potentiometer. Let me know if you need modifications. Shall I proceed to **Q8**?

## Q8: Design the Circuit Using LED and Make a Counter Till 20 and Display on the LED Screen.

---

### Required Components:

- **Arduino Uno**
  - **LED Display (Common Cathode or Anode 7-segment display)**
  - **Jumper Wires**
  - **Resistors (220Ω)**
  - **Breadboard**
- 

### Circuit Connections:

For simplicity, we'll use a **7-segment display** to display the counter.

### 7-Segment Display Connections:

If you're using a **common cathode** display, the cathode pins (common pins) will go to **GND**. Each segment (A, B, C, etc.) is connected to individual **Arduino pins** through current-limiting resistors (220Ω).

Here is a simple mapping for a **7-segment display**:

- **A → Pin 2**
  - **B → Pin 3**
  - **C → Pin 4**
  - **D → Pin 5**
  - **E → Pin 6**
  - **F → Pin 7**
  - **G → Pin 8**
  - **Common Cathode Pin → GND**
- 

### Full Working Code:

cpp

CopyEdit

```
int segments[] = {2, 3, 4, 5, 6, 7, 8}; // Pins connected to segments A-G
```

```
// Digits represented by segments (A-G)
```

```
byte digit[] = {
```

```

B11111100, // 0
B01100000, // 1
B11011010, // 2
B11110010, // 3
B01100110, // 4
B10110110, // 5
B10111110, // 6
B11100000, // 7
B11111110, // 8
B11110110 // 9
};

void setup() {
    // Set all segment pins as output
    for (int i = 0; i < 7; i++) {
        pinMode(segments[i], OUTPUT);
    }
}

void displayDigit(int num) {
    byte segmentsState = digit[num]; // Get the corresponding segment pattern
    for (int i = 0; i < 7; i++) {
        // If the bit is 1, turn on the corresponding segment
        if (segmentsState & (1 << i)) {
            digitalWrite(segments[i], HIGH);
        } else {
            digitalWrite(segments[i], LOW);
        }
    }
}

```

```
void loop() {  
  for (int i = 0; i <= 20; i++) {  
    displayDigit(i % 10); // Display the digit, modulus for cycling 0-9  
    delay(1000); // 1-second delay between each digit  
  }  
}
```

---

**Working:**

1. **The counter will increment from 0 to 9 on the 7-segment display.**
  2. **After 9, it will reset to 0 and repeat the counting till 20.**
  3. **The digits will be displayed sequentially, with a delay of 1 second for each number.**
- 

This setup shows the numbers on a **7-segment display** from **0 to 9** and then loops back. Would you like me to proceed with **Q9**?

4o mini

**You said:**



## Q9: Implement a Traffic Light Using 3 LEDs and Arduino System. It Should Mimic the Real World Condition.

---

### Required Components:

- **Arduino Uno**
  - **3 LEDs** (Red, Yellow, Green)
  - **3 Resistors** ( $220\Omega$ )
  - **Breadboard**
  - **Jumper Wires**
- 

### Circuit Connections:

#### LED Connections:

- **Red LED:**
  - Anode (long leg) → Pin **2** on Arduino
  - Cathode (short leg) → **Ground (GND)** through a  **$220\Omega$  resistor**
- **Yellow LED:**
  - Anode → Pin **3** on Arduino
  - Cathode → **Ground (GND)** through a  **$220\Omega$  resistor**
- **Green LED:**
  - Anode → Pin **4** on Arduino
  - Cathode → **Ground (GND)** through a  **$220\Omega$  resistor**

#### LCD (Optional, for Displaying Traffic Light Status):

- **Pins for LCD:**
  - **RS** → Pin **12**
  - **E** → Pin **11**
  - **D4** → Pin **5**
  - **D5** → Pin **4**
  - **D6** → Pin **3**
  - **D7** → Pin **2**
  - **VSS** → **GND**
  - **VDD** → **5V**

- **RW → GND**

---

### Full Working Code:

cpp

CopyEdit

```
#include <LiquidCrystal.h>
```

```
// Pin setup for the LEDs
```

```
int redPin = 2;
```

```
int yellowPin = 3;
```

```
int greenPin = 4;
```

```
// Setup the LCD (adjust the pins according to your setup)
```

```
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

```
void setup() {
```

```
    // Initialize the LED pins as outputs
```

```
    pinMode(redPin, OUTPUT);
```

```
    pinMode(yellowPin, OUTPUT);
```

```
    pinMode(greenPin, OUTPUT);
```

```
    // Initialize the LCD
```

```
    lcd.begin(16, 2);
```

```
    lcd.print("Traffic Light");
```

```
}
```

```
void loop() {
```

```
    // Red light for 5 seconds
```

```
    digitalWrite(redPin, HIGH); // Turn on red LED
```

```
    digitalWrite(yellowPin, LOW); // Turn off yellow LED
```

```
    digitalWrite(greenPin, LOW); // Turn off green LED
```

```
lcd.setCursor(0, 1);  
lcd.print("Red Light  ");  
delay(5000); // Wait for 5 seconds  
  
// Yellow light for 2 seconds  
digitalWrite(redPin, LOW); // Turn off red LED  
digitalWrite(yellowPin, HIGH); // Turn on yellow LED  
digitalWrite(greenPin, LOW); // Turn off green LED  
lcd.setCursor(0, 1);  
lcd.print("Yellow Light ");  
delay(2000); // Wait for 2 seconds  
  
// Green light for 5 seconds  
digitalWrite(redPin, LOW); // Turn off red LED  
digitalWrite(yellowPin, LOW); // Turn off yellow LED  
digitalWrite(greenPin, HIGH); // Turn on green LED  
lcd.setCursor(0, 1);  
lcd.print("Green Light ");  
delay(5000); // Wait for 5 seconds  
}
```

---

### Working Explanation:

1. **Red Light:** The red LED lights up for 5 seconds.
2. **Yellow Light:** The yellow LED lights up for 2 seconds.
3. **Green Light:** The green LED lights up for 5 seconds.
4. **LCD Display:** The LCD will display the traffic light status (Red Light, Yellow Light, or Green Light) on the second row.
5. The sequence repeats indefinitely, simulating a real-world traffic light cycle.

---

### Additional Notes:

- You can adjust the timing for the red, yellow, and green lights by modifying the delay values.

- The traffic light system will cycle through red, yellow, and green repeatedly, just like a real traffic light.

Would you like to proceed to **Q10**?

## Q10: Use LDR Sensor and LEDs. Design the Circuit to Glow the LED at Various Different Values with Serial Monitor Display.

---

### Required Components:

- **Arduino Uno**
  - **LDR (Light Dependent Resistor)**
  - **1 LED** (with a 220Ω resistor)
  - **10kΩ Resistor** (for LDR voltage divider)
  - **Breadboard**
  - **Jumper Wires**
- 

### Circuit Connections:

#### LDR and Voltage Divider:

1. **LDR:**
  - One leg of the LDR → **5V** on the Arduino
  - The other leg of the LDR → **Analog Pin A0** on the Arduino and connect a **10kΩ resistor** from the same point to **Ground (GND)**

#### LED Connection:

2. **LED:**
    - Anode (long leg) → **Pin 3** on Arduino
    - Cathode (short leg) → **Ground (GND)** through a **220Ω resistor**
- 

### Full Working Code:

cpp

CopyEdit

```
int ldrPin = A0; // LDR connected to analog pin A0
```

```
int ledPin = 3; // LED connected to digital pin 3
```

```
int ldrValue = 0; // Variable to store the LDR value
```

```
void setup() {
```

```
    pinMode(ledPin, OUTPUT); // Set LED pin as output
```

```
    Serial.begin(9600); // Start serial communication
```

```
}

void loop() {
  // Read the LDR value

  ldrValue = analogRead(ldrPin); // Read the value from the LDR (0-1023)


  // Display LDR value in Serial Monitor
  Serial.print("LDR Value: ");
  Serial.println(ldrValue);


  // Map the LDR value to LED brightness (0-255)
  int brightness = map(ldrValue, 0, 1023, 0, 255);


  // Control the LED brightness based on LDR value
  analogWrite(ledPin, brightness); // Use PWM to adjust brightness


  delay(500); // Wait for 500ms before taking the next reading
}
```

---

### Working Explanation:

1. **LDR (Light Dependent Resistor):** The LDR's resistance changes based on the light intensity it receives. The more light it gets, the lower its resistance.
2. **Voltage Divider:** The LDR is used in a voltage divider configuration with a **10kΩ resistor**. This creates a voltage that varies with the light intensity, which is read by the Arduino via the **analog input pin A0**.
3. **LED Control:** Based on the LDR value, the brightness of the LED will change using **PWM (Pulse Width Modulation)**. The higher the light intensity, the brighter the LED becomes.
4. **Serial Monitor:** The LDR value is printed on the Serial Monitor to show the changes in light intensity and LED brightness.

---

### Additional Notes:

- **LDR:** When there is more light, the resistance of the LDR decreases, resulting in a higher voltage at the analog input pin of the Arduino. When the light level is low, the LDR's resistance increases, and the voltage at the analog pin decreases.
- **LED:** The LED brightness is controlled through **analogWrite()**, which uses PWM to adjust the brightness between 0 (off) and 255 (fully on).

Would you like to proceed with **Q11**?

## Q11: Use Ultrasonic Sensor and 03 LEDs. Design the Circuit to Glow the LEDs at Three Different Distances with Its Display on Serial Monitor.

---

### Required Components:

- **Arduino Uno**
  - **Ultrasonic Sensor (HC-SR04)**
  - **3 LEDs** (Red, Yellow, Green)
  - **220Ω Resistors** for each LED
  - **Breadboard**
  - **Jumper Wires**
- 

### Circuit Connections:

#### Ultrasonic Sensor (HC-SR04):

1. **VCC** pin of the ultrasonic sensor → **5V** on Arduino
2. **GND** pin of the ultrasonic sensor → **Ground (GND)** on Arduino
3. **Trig** pin of the ultrasonic sensor → **Pin 9** on Arduino
4. **Echo** pin of the ultrasonic sensor → **Pin 10** on Arduino

#### LEDs:

1. **Red LED:**
    - Anode (long leg) → **Pin 3** on Arduino
    - Cathode (short leg) → **Ground (GND)** through a **220Ω resistor**
  2. **Yellow LED:**
    - Anode (long leg) → **Pin 4** on Arduino
    - Cathode (short leg) → **Ground (GND)** through a **220Ω resistor**
  3. **Green LED:**
    - Anode (long leg) → **Pin 5** on Arduino
    - Cathode (short leg) → **Ground (GND)** through a **220Ω resistor**
- 

### Full Working Code:

cpp

CopyEdit



```
#define TRIG_PIN 9 // Trigger pin of ultrasonic sensor
#define ECHO_PIN 10 // Echo pin of ultrasonic sensor

#define RED_LED 3 // Red LED connected to pin 3
#define YELLOW_LED 4 // Yellow LED connected to pin 4
#define GREEN_LED 5 // Green LED connected to pin 5

long duration;
int distance;

void setup() {
  pinMode(TRIG_PIN, OUTPUT); // Set the trig pin as output
  pinMode(ECHO_PIN, INPUT); // Set the echo pin as input
  pinMode(RED_LED, OUTPUT); // Set Red LED as output
  pinMode(YELLOW_LED, OUTPUT); // Set Yellow LED as output
  pinMode(GREEN_LED, OUTPUT); // Set Green LED as output

  Serial.begin(9600); // Start serial communication
}

void loop() {
  // Send a 10 microsecond pulse to trigger the ultrasonic sensor
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // Measure the time it takes for the pulse to return
  duration = pulseIn(ECHO_PIN, HIGH);
```

```

// Calculate the distance in centimeters

distance = duration * 0.0344 / 2;


// Display the distance on the Serial Monitor
Serial.print("Distance: ");
Serial.print(distance);
Serial.println(" cm");


// Control the LEDs based on the distance
if (distance < 10) {
    digitalWrite(REDA_LED, HIGH); // Red LED ON for very close distance
    digitalWrite(YELLOW_LED, LOW);
    digitalWrite(GREEN_LED, LOW);
}
else if (distance >= 10 && distance < 20) {
    digitalWrite(REDA_LED, LOW);
    digitalWrite(YELLOW_LED, HIGH); // Yellow LED ON for medium distance
    digitalWrite(GREEN_LED, LOW);
}
else {
    digitalWrite(REDA_LED, LOW);
    digitalWrite(YELLOW_LED, LOW);
    digitalWrite(GREEN_LED, HIGH); // Green LED ON for far distance
}

delay(500); // Wait for half a second before taking the next reading
}

```

---

### Working Explanation:

#### 1. Ultrasonic Sensor (HC-SR04):

- The sensor sends out a pulse when the **Trig** pin is triggered and measures how long it takes for the pulse to bounce back from an object, which is captured on the **Echo** pin.
- This pulse duration is converted into distance using the speed of sound formula:  

$$\text{Distance} = \text{Duration} \times 0.03442$$

$$\text{Distance} = \frac{\text{Duration}}{2} \times 0.0344$$
- The sensor is capable of measuring distances from 2 cm to 400 cm.

## 2. LED Control:

- **Red LED:** Turns on if the distance is less than 10 cm, indicating the object is very close.
- **Yellow LED:** Turns on if the distance is between 10 cm and 20 cm, indicating the object is at a medium distance.
- **Green LED:** Turns on if the distance is greater than 20 cm, indicating the object is far away.

## 3. Serial Monitor:

- The distance value is continuously displayed on the **Serial Monitor** for tracking the measured distance.

## 4. LED Display:

- The LEDs indicate the proximity of an object:
  - **Red** for close,
  - **Yellow** for medium distance,
  - **Green** for far distance.

---

## Additional Notes:

- **Ultrasonic Sensor's Range:** It works by measuring the time taken by sound waves to travel to an object and return. The sensor is accurate for distances between 2 cm and 400 cm.
- **LED Control:** The LEDs light up based on predefined distance thresholds, which are easily adjustable in the code.

Would you like to proceed with **Q12**?

**Q12: Design the Circuit for a Servo Motor with a Potentiometer. It Is Expected That the Servo Motor Moves with Change in Potentiometer Position. Display the Digital Values in the Serial Monitor.**

---

**Required Components:**

- **Arduino Uno**
  - **Servo Motor**
  - **Potentiometer**
  - **Breadboard**
  - **Jumper Wires**
- 

**Circuit Connections:**

**Servo Motor:**

1. **VCC** pin of the servo → **5V** on Arduino
2. **GND** pin of the servo → **Ground (GND)** on Arduino
3. **Signal (PWM) pin** of the servo → **Pin 9** on Arduino

**Potentiometer:**

1. **VCC** pin (one side) of the potentiometer → **5V** on Arduino
  2. **GND** pin (other side) of the potentiometer → **Ground (GND)** on Arduino
  3. **Wiper (middle pin)** of the potentiometer → **Pin A0** on Arduino (Analog Input)
- 

**Full Working Code:**

cpp

CopyEdit

```
#include <Servo.h>
```

```
Servo myServo; // Create a Servo object to control the servo motor
```

```
int potPin = A0; // Pin where the potentiometer is connected
```

```
int val = 0; // Variable to store potentiometer value (0-1023)
```

```
int angle = 0; // Variable to store the corresponding angle for the servo
```

```
void setup() {  
    myServo.attach(9); // Attach the servo motor to pin 9 on Arduino  
    Serial.begin(9600); // Start serial communication for debugging  
}  
  
void loop() {  
    // Read the value from the potentiometer (0-1023)  
    val = analogRead(potPin);  
  
    // Map the potentiometer value to a servo angle (0-180 degrees)  
    angle = map(val, 0, 1023, 0, 180);  
  
    // Move the servo motor to the mapped angle  
    myServo.write(angle);  
  
    // Print the potentiometer value and corresponding angle to the serial monitor  
    Serial.print("Potentiometer Value: ");  
    Serial.print(val);  
    Serial.print("\tServo Angle: ");  
    Serial.println(angle);  
  
    delay(15); // Wait for the servo to reach the position  
}
```

---

### **Working Explanation:**

#### **1. Potentiometer:**

- A potentiometer is used to generate a variable voltage, which is read as an analog value by the Arduino.
- The potentiometer provides a voltage between 0 and 5V, corresponding to values between 0 and 1023 when read by `analogRead()`.

#### **2. Mapping Potentiometer Values to Servo Angles:**

- The potentiometer value (0-1023) is mapped to the range 0-180, which corresponds to the servo's rotational range. The map() function is used for this purpose:

cpp

CopyEdit

```
angle = map(val, 0, 1023, 0, 180);
```

- This ensures that when the potentiometer is at its minimum value (0), the servo angle will be 0 degrees (full left), and when the potentiometer is at its maximum value (1023), the servo will move to 180 degrees (full right).

### 3. Servo Control:

- The servo is moved to the desired angle using the write() method:

cpp

CopyEdit

```
myServo.write(angle);
```

### 4. Serial Monitor:

- The potentiometer value and the corresponding servo angle are displayed on the serial monitor for debugging and tracking the position of the servo motor.

---

### Additional Notes:

- The servo motor will move smoothly based on the potentiometer's position.
- The delay(15) ensures that the servo has enough time to reach the new position before reading the potentiometer again.
- You can adjust the range of servo movement by changing the map() function's values, but a typical servo motor moves within 0 to 180 degrees.

Would you like to proceed with **Q13**?

**Q13: Design the Circuit for an Infrared Sensor. Use Two or Three LEDs to Glow at Different Values. Display the Value on the Serial Monitor.**

---

**Required Components:**

- **Arduino Uno**
  - **Infrared (IR) Sensor**
  - **LEDs (2 or 3 LEDs)**
  - **Resistors (220 ohms)**
  - **Breadboard**
  - **Jumper Wires**
- 

**Circuit Connections:**

**Infrared Sensor:**

1. **VCC** pin of the IR sensor → **5V** on Arduino
2. **GND** pin of the IR sensor → **Ground (GND)** on Arduino
3. **OUT** pin of the IR sensor → **Pin 8** on Arduino (Digital Input)

**LEDs:**

- **LED 1 (e.g., Red):**
    - **Anode (long leg)** → **Pin 3** on Arduino (Digital Output)
    - **Cathode (short leg)** → **Ground (GND)** with a **220-ohm resistor** in series.
  - **LED 2 (e.g., Green):**
    - **Anode (long leg)** → **Pin 4** on Arduino (Digital Output)
    - **Cathode (short leg)** → **Ground (GND)** with a **220-ohm resistor** in series.
  - **LED 3 (e.g., Yellow) (optional):**
    - **Anode (long leg)** → **Pin 5** on Arduino (Digital Output)
    - **Cathode (short leg)** → **Ground (GND)** with a **220-ohm resistor** in series.
- 

**Full Working Code:**

cpp

CopyEdit

```
#define IR_SENSOR_PIN 8 // Pin connected to the output of the IR sensor
```

```

#define LED1 3    // Red LED connected to Pin 3
#define LED2 4    // Green LED connected to Pin 4
#define LED3 5    // Yellow LED connected to Pin 5 (optional)

void setup() {
  pinMode(IR_SENSOR_PIN, INPUT); // Set IR sensor pin as input
  pinMode(LED1, OUTPUT);        // Set Red LED pin as output
  pinMode(LED2, OUTPUT);        // Set Green LED pin as output
  pinMode(LED3, OUTPUT);        // Set Yellow LED pin as output

  Serial.begin(9600); // Start serial communication for debugging
}

void loop() {
  int sensorValue = digitalRead(IR_SENSOR_PIN); // Read the sensor value

  // Display the value of the sensor on the serial monitor
  Serial.print("Sensor Value: ");
  Serial.println(sensorValue);

  // Based on the sensor value, turn on the appropriate LED
  if (sensorValue == LOW) { // If IR sensor detects an object (LOW means object detected)
    digitalWrite(LED1, HIGH); // Turn on Red LED
    digitalWrite(LED2, LOW);  // Turn off Green LED
    digitalWrite(LED3, LOW);  // Turn off Yellow LED
  }
  else { // If no object is detected (sensor reads HIGH)
    digitalWrite(LED1, LOW); // Turn off Red LED
    digitalWrite(LED2, HIGH); // Turn on Green LED
    digitalWrite(LED3, LOW);  // Turn off Yellow LED
  }
}

```



```
delay(500); // Delay to make the LED changes visible  
}
```

---

### Working Explanation:

#### 1. IR Sensor:

- The **infrared sensor** detects the presence of an object in front of it. When an object is detected, the output pin of the IR sensor goes **LOW**, and when no object is detected, the output pin goes **HIGH**.
- The `digitalRead()` function is used to read the value of the IR sensor pin.

#### 2. LEDs:

- **LED 1 (Red)** lights up when the IR sensor detects an object (sensor value **LOW**).
- **LED 2 (Green)** lights up when the IR sensor does not detect an object (sensor value **HIGH**).
- **LED 3 (Yellow)** is optional and can be used for additional indication if needed.

#### 3. Serial Monitor:

- The **sensor value** (either **HIGH** or **LOW**) is printed on the serial monitor. This allows you to track when the sensor detects an object.

#### 4. LED Behavior:

- When the IR sensor detects an object, the **Red LED** lights up.
  - When no object is detected, the **Green LED** lights up.
  - If you want, you can add more logic for the **Yellow LED** or modify its behavior based on the sensor value or other conditions.
- 

### Additional Notes:

- The **IR sensor** typically outputs a **LOW** signal when it detects an object and a **HIGH** signal when it doesn't.
- You can adjust the LED behavior based on the sensor's input or add more conditions for other LEDs as required.

Would you like to proceed with **Q14**?

**Q14: Design the Circuit for a DC Motor with Relay. It Is Expected to Glow the Two LEDs to Represent ON/OFF Status of Motor.**

---

**Required Components:**

- **Arduino Uno**
  - **DC Motor**
  - **Relay Module**
  - **LEDs (2 LEDs: one for indicating ON status, and one for OFF status)**
  - **Diode (e.g., 1N4007)**
  - **Resistors (220 ohms for LEDs)**
  - **Power Supply (for the motor)**
  - **Breadboard**
  - **Jumper Wires**
- 

**Circuit Connections:**

**DC Motor and Relay:**

1. **DC Motor:**
  - Connect the **positive terminal** of the DC motor to the **NO (Normally Open)** terminal of the relay.
  - Connect the **negative terminal** of the DC motor to the **Ground (GND)** of the motor's power supply.
2. **Relay Module:**
  - **VCC** of the relay module → **5V** on Arduino
  - **GND** of the relay module → **Ground (GND)** on Arduino
  - **IN (Input)** pin of the relay module → **Pin 7** on Arduino (Digital Output)
  - **COM (Common)** terminal of the relay → **positive terminal** of the DC motor power supply.

**LEDs (ON/OFF Status):**

- **LED 1 (Green - Motor ON):**
  - **Anode (long leg)** → **Pin 3** on Arduino (Digital Output)
  - **Cathode (short leg)** → **Ground (GND)** with a **220-ohm resistor** in series.
- **LED 2 (Red - Motor OFF):**

- **Anode (long leg) → Pin 4** on Arduino (Digital Output)
- **Cathode (short leg) → Ground (GND)** with a **220-ohm resistor** in series.

#### Power Supply:

- Provide a **separate power supply** for the DC motor (usually 6V or 12V, depending on your motor's requirements). Make sure the **GND of the power supply** is connected to the **GND of Arduino** for a common ground reference.

---

#### Full Working Code:

cpp

CopyEdit

```
#define MOTOR_RELAY_PIN 7 // Pin connected to the relay input (to control motor)
```

```
#define LED_ON_PIN 3 // Green LED for motor ON
```

```
#define LED_OFF_PIN 4 // Red LED for motor OFF
```

```
void setup() {
```

```
    pinMode(MOTOR_RELAY_PIN, OUTPUT); // Set relay pin as output
```

```
    pinMode(LED_ON_PIN, OUTPUT); // Set green LED pin as output
```

```
    pinMode(LED_OFF_PIN, OUTPUT); // Set red LED pin as output
```

```
    // Initially, motor is off, and red LED is on
```

```
    digitalWrite(MOTOR_RELAY_PIN, LOW); // Relay off (motor off)
```

```
    digitalWrite(LED_ON_PIN, LOW); // Green LED off
```

```
    digitalWrite(LED_OFF_PIN, HIGH); // Red LED on
```

```
    Serial.begin(9600); // Start serial communication for debugging
```

```
}
```

```
void loop() {
```

```
    // Simulate motor ON by activating the relay and LEDs
```

```
    digitalWrite(MOTOR_RELAY_PIN, HIGH); // Relay on (motor on)
```

```
    digitalWrite(LED_ON_PIN, HIGH); // Green LED on (motor running)
```

```
    digitalWrite(LED_OFF_PIN, LOW); // Red LED off (motor running)
```

```
Serial.println("Motor ON"); // Print status to Serial Monitor

delay(5000); // Motor stays on for 5 seconds

// Simulate motor OFF by deactivating the relay and LEDs
digitalWrite(MOTOR_RELAY_PIN, LOW); // Relay off (motor off)
digitalWrite(LED_ON_PIN, LOW);    // Green LED off (motor stopped)
digitalWrite(LED_OFF_PIN, HIGH);  // Red LED on (motor stopped)

Serial.println("Motor OFF"); // Print status to Serial Monitor

delay(5000); // Motor stays off for 5 seconds
}
```

---

### Working Explanation:

#### 1. DC Motor Control via Relay:

- The **relay module** is used to control the motor. When the relay is activated (using **digitalWrite(MOTOR\_RELAY\_PIN, HIGH)**), the motor turns ON. When the relay is deactivated (using **digitalWrite(MOTOR\_RELAY\_PIN, LOW)**), the motor turns OFF.

#### 2. LED Status Indicators:

- **Green LED (LED 1)** lights up when the motor is ON, indicating the motor is running.
- **Red LED (LED 2)** lights up when the motor is OFF, indicating the motor is stopped.

#### 3. Serial Monitor:

- The serial monitor displays the motor's status (ON/OFF), which helps to debug and observe the system in real-time.

#### 4. Relay Pin:

- The relay pin is controlled by **digitalWrite(MOTOR\_RELAY\_PIN, HIGH)** or **LOW** to switch the motor ON or OFF.

#### 5. Delays:

- The motor remains ON for 5 seconds and then OFF for 5 seconds in the **loop()** function. You can adjust the delay time to suit your requirements.
- 

**Additional Notes:**

- Ensure you use a **diode (1N4007)** across the relay's **NO** and **COM** terminals to prevent back-emf from damaging the relay and Arduino.
- The **Relay** is used because the DC motor often requires more current than the Arduino can safely supply, and the relay isolates the motor's power supply from the Arduino.
- Adjust the **motor power supply** voltage and current based on the specifications of your DC motor.

Would you like to continue with **Q15**?

### Q15: Design the Circuit Using Potentiometer, and LCD Display and Scroll 'YOUR NAME' on LCD Screen.

---

#### Required Components:

- **Arduino Uno**
  - **Potentiometer** (10k $\Omega$  is a common value)
  - **16x2 LCD Display with I2C Interface**
  - **Jumper Wires**
  - **Breadboard**
- 

#### Circuit Connections:

1. **Potentiometer:**
    - **One end of the potentiometer** → **5V on Arduino**
    - **Other end of the potentiometer** → **Ground (GND) on Arduino**
    - **Wiper (middle pin)** of the potentiometer → **A0** (Analog Input Pin) on Arduino
  2. **LCD Display:**
    - **VCC** of the LCD → **5V on Arduino**
    - **GND** of the LCD → **Ground (GND) on Arduino**
    - **SDA** of the LCD → **A4** on Arduino (Data Line)
    - **SCL** of the LCD → **A5** on Arduino (Clock Line)
- 

#### Full Working Code:

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD address 0x27, 16 columns, 2 rows
```

```
#define POT_PIN A0 // Potentiometer connected to A0
```

```

void setup() {
    // Initialize the LCD and set backlight
    lcd.begin(16, 2);
    lcd.backlight();

    // Display an initial message on the LCD
    lcd.setCursor(0, 0);
    lcd.print("Scrolling Name:");

    // Wait for a moment before starting the scroll
    delay(2000);
}

void loop() {
    int potValue = analogRead(POT_PIN); // Read potentiometer value (0 to 1023)
    int scrollSpeed = map(potValue, 0, 1023, 50, 500); // Map potentiometer to scroll speed

    String name = "YOUR NAME"; // Name to scroll

    // Scroll the name across the screen
    for (int i = 0; i < name.length(); i++) {
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print(name.substring(i)); // Display substring starting from the current position
        lcd.print(name.substring(0, i)); // Display the part of name from start to current position

        delay(scrollSpeed); // Delay based on potentiometer value (speed control)
    }
}

```

---

**Working Explanation:**

### 1. LCD Initialization:

- The **LiquidCrystal\_I2C** library is used to control the LCD with I2C. The LCD is initialized with the address 0x27 (common I2C address for these LCDs) and a size of 16x2 (16 columns and 2 rows).

### 2. Potentiometer Control:

- The potentiometer is connected to the **A0 pin** (analog input). The value from the potentiometer is read using **analogRead(POT\_PIN)**, which provides a value between 0 and 1023. This value is mapped to a range of **50 to 500 milliseconds** to control the speed of the scrolling text.

### 3. Text Scrolling:

- The **String name** variable holds the text to be scrolled, which is "YOUR NAME".
- The scrolling is done by repeatedly clearing the screen, displaying a substring of the name starting from the current position, and displaying the remaining portion of the name.
- **delay(scrollSpeed)** controls how fast the text scrolls based on the potentiometer's position.

### 4. Speed Control:

- The potentiometer's value is mapped using **map()** to adjust the speed of scrolling. A higher potentiometer value means slower scrolling, and a lower value results in faster scrolling.

---

### Additional Notes:

- **Potentiometer Adjustment:** By turning the potentiometer, you can adjust the speed at which the text scrolls on the LCD.
- **LCD Display:** The text will scroll horizontally across the 16-character wide screen. If the text is longer than 16 characters, it will wrap around and scroll again.
- **Power Supply:** The Arduino will power the LCD and potentiometer through its 5V pin.

Would you like to continue with **Q16**?



## Q16: Design the Circuit Using LED and Make a Counter Till 20 and Display on the LED Screen.

---

### Required Components:

- **Arduino Uno**
  - **LED (1 LED)**
  - **220Ω Resistor** (for current limiting)
  - **16x2 LCD Display with I2C Interface**
  - **Jumper Wires**
  - **Breadboard**
- 

### Circuit Connections:

1. **LED:**
    - **Long leg (Anode)** of the LED → **Digital Pin 13** on Arduino
    - **Short leg (Cathode)** of the LED → **220Ω Resistor** → **Ground (GND)** on Arduino
  2. **LCD Display:**
    - **VCC** of the LCD → **5V on Arduino**
    - **GND** of the LCD → **Ground (GND) on Arduino**
    - **SDA** of the LCD → **A4** on Arduino (Data Line)
    - **SCL** of the LCD → **A5** on Arduino (Clock Line)
- 

### Full Working Code:

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD address 0x27, 16 columns, 2 rows
```

```
#define LED_PIN 13 // LED connected to digital pin 13
```

```
void setup() {  
    // Initialize the LCD  
    lcd.begin(16, 2);  
    lcd.backlight();  
  
    // Initialize LED pin as output  
    pinMode(LED_PIN, OUTPUT);  
  
    // Display initial message  
    lcd.setCursor(0, 0);  
    lcd.print("Counter: 0");  
    delay(1000); // Display for 1 second  
}  
  
void loop() {  
    for (int count = 1; count <= 20; count++) {  
        // Turn on the LED  
        digitalWrite(LED_PIN, HIGH);  
  
        // Update LCD with the current counter value  
        lcd.clear();  
        lcd.setCursor(0, 0);  
        lcd.print("Counter: ");  
        lcd.print(count);  
  
        delay(1000); // Wait for 1 second  
  
        // Turn off the LED  
        digitalWrite(LED_PIN, LOW);  
  
        delay(500); // Wait for 0.5 second before next count
```

```

}

// After counter reaches 20, display 'Completed'
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Completed!");
delay(2000); // Display for 2 seconds

// Reset the counter to start again
for (int count = 1; count <= 20; count++) {
    // Reset the display for the next round of counting
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Counter: ");
    lcd.print(count);
    delay(1000);
}
}

```

---

### Working Explanation:

#### 1. LCD Initialization:

- The **LiquidCrystal\_I2C** library is used to control the LCD with I2C. The LCD is initialized with the address 0x27 (common I2C address for these LCDs) and a size of 16x2 (16 columns and 2 rows).

#### 2. LED Pin Setup:

- The LED is connected to **digital pin 13** on Arduino. It is set as an **output** pin in the setup() function using pinMode(LED\_PIN, OUTPUT).

#### 3. Counter Logic:

- A for loop runs from **1 to 20**, and with each count, the LED is turned **ON** using digitalWrite(LED\_PIN, HIGH).
- The LCD is updated to show the current counter value using lcd.setCursor() to place the cursor at the appropriate location and lcd.print() to display the number.

- After each count, the LED is turned **OFF** using `digitalWrite(LED_PIN, LOW)` for half a second to create a flashing effect. The loop continues until the counter reaches 20.

#### 4. Completion Display:

- Once the counter reaches **20**, the message "Completed!" is displayed on the LCD for 2 seconds.
- The program then resets the counter and repeats the counting process.

---

#### Additional Notes:

- **LED Behavior:** The LED will turn ON for each count and then OFF for half a second. This simulates a flashing effect along with the counter.
- **LCD Display:** The LCD displays the current count, updating every second. After counting up to 20, it shows "Completed!" for 2 seconds.
- **Looping Behavior:** The counter will continue looping until you reset the Arduino. If you want to stop the counting, you can add a condition or button input to halt the process.

---

Would you like to proceed with **Q17**?

## Q17: Implement a Traffic Light Using 3 LEDs and Arduino System. It Should Mimic the Real-World Condition.

---

### Required Components:

- **Arduino Uno**
  - **3 LEDs** (Red, Yellow, Green)
  - **220Ω Resistors** (for current limiting)
  - **Jumper Wires**
  - **Breadboard**
  - **16x2 LCD Display with I2C Interface**
- 

### Circuit Connections:

#### 1. LEDs:

- **Red LED:** Long leg (Anode) to **Digital Pin 3**, Short leg (Cathode) to **220Ω Resistor → Ground (GND)**.
- **Yellow LED:** Long leg (Anode) to **Digital Pin 4**, Short leg (Cathode) to **220Ω Resistor → Ground (GND)**.
- **Green LED:** Long leg (Anode) to **Digital Pin 5**, Short leg (Cathode) to **220Ω Resistor → Ground (GND)**.

#### 2. LCD Display:

- **VCC** of the LCD → **5V** on Arduino.
  - **GND** of the LCD → **Ground (GND)** on Arduino.
  - **SDA** of the LCD → **A4** on Arduino (Data Line).
  - **SCL** of the LCD → **A5** on Arduino (Clock Line).
- 

### Full Working Code:

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD address 0x27, 16 columns, 2 rows
```

```
// LED pins

#define RED_PIN 3

#define YELLOW_PIN 4

#define GREEN_PIN 5


void setup() {

    // Initialize the LCD

    lcd.begin(16, 2);

    lcd.backlight();


    // Initialize LED pins as OUTPUT
    pinMode(RED_PIN, OUTPUT);
    pinMode(YELLOW_PIN, OUTPUT);
    pinMode(GREEN_PIN, OUTPUT);


    // Display initial message
    lcd.setCursor(0, 0);
    lcd.print("Traffic Light");
    delay(1000); // Display for 1 second
}


void loop() {

    // Red light ON (STOP)
    digitalWrite(RED_PIN, HIGH); // Red LED ON
    digitalWrite(YELLOW_PIN, LOW); // Yellow LED OFF
    digitalWrite(GREEN_PIN, LOW); // Green LED OFF


    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("Red: STOP");
```

```
delay(5000); // Red light stays for 5 seconds

// Yellow light ON (GET READY)
digitalWrite(RED_PIN, LOW); // Red LED OFF
digitalWrite(YELLOW_PIN, HIGH); // Yellow LED ON
digitalWrite(GREEN_PIN, LOW); // Green LED OFF

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Yellow: READY");
delay(2000); // Yellow light stays for 2 seconds

// Green light ON (GO)
digitalWrite(RED_PIN, LOW); // Red LED OFF
digitalWrite(YELLOW_PIN, LOW); // Yellow LED OFF
digitalWrite(GREEN_PIN, HIGH); // Green LED ON

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Green: GO");
delay(5000); // Green light stays for 5 seconds
}
```

---

### Working Explanation:

#### 1. LCD Initialization:

- The **LiquidCrystal\_I2C** library is used to control the LCD. The LCD is initialized with address 0x27 (common I2C address) and a 16x2 size.

#### 2. LED Pins Setup:

- The Red, Yellow, and Green LEDs are connected to **digital pins 3, 4, and 5** respectively.
- All LED pins are set as **OUTPUT** in the setup() function.

#### 3. Traffic Light Simulation:

- In the loop() function, the following sequence happens:
  - **Red light ON:** The Red LED is turned ON for **5 seconds**, while Yellow and Green LEDs are OFF.
  - **Yellow light ON:** After the Red light, the Yellow LED is turned ON for **2 seconds** while Red and Green LEDs are OFF.
  - **Green light ON:** Finally, the Green LED is turned ON for **5 seconds**, while Red and Yellow LEDs are OFF.

#### 4. LCD Display Updates:

- The LCD displays the current light status ("Red: STOP", "Yellow: READY", "Green: GO") during each stage, and the corresponding LED is lit for the specified time.

---

#### Additional Notes:

- **LED Timing:** The timing for each light is set using the delay() function. The Red and Green lights stay ON for 5 seconds, while the Yellow light stays ON for 2 seconds.
- **LCD Updates:** The LCD is updated every time the light changes to reflect the current traffic signal status.

---

Would you like to proceed with **Q18** next?



**Q18: Use LDR Sensor and LEDs. Design the Circuit to Glow the LED at Various Different Values with Serial Monitor Display.**

---

**Required Components:**

- **Arduino Uno**
  - **LDR (Light Dependent Resistor)**
  - **1k $\Omega$  Resistor**
  - **LED** (any color)
  - **220 $\Omega$  Resistor** (for current limiting)
  - **Jumper Wires**
  - **Breadboard**
  - **16x2 LCD Display with I2C Interface** (optional for display)
- 

**Circuit Connections:**

**1. LDR (Light Dependent Resistor):**

- Connect one leg of the LDR to **5V**.
- Connect the other leg to **Analog Pin A0** and also connect it to one leg of the **1k $\Omega$  resistor**.
- The other leg of the **1k $\Omega$  resistor** goes to **Ground (GND)**. This forms a voltage divider to read the light level.

**2. LED:**

- Connect the **Anode** (long leg) of the LED to **Digital Pin 9**.
- Connect the **Cathode** (short leg) of the LED to **Ground (GND)** through the **220 $\Omega$  resistor**.

**3. LCD Display (Optional):**

- **VCC** of the LCD  $\rightarrow$  **5V** on Arduino.
  - **GND** of the LCD  $\rightarrow$  **Ground (GND)** on Arduino.
  - **SDA** of the LCD  $\rightarrow$  **A4** on Arduino (Data Line).
  - **SCL** of the LCD  $\rightarrow$  **A5** on Arduino (Clock Line).
- 

**Full Working Code:**

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
// LCD initialization
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD address 0x27, 16 columns, 2 rows
```

```
// LED Pin
```

```
#define LED_PIN 9
```

```
// LDR Pin
```

```
#define LDR_PIN A0
```

```
void setup() {
```

```
    // Initialize the LCD
```

```
    lcd.begin(16, 2);
```

```
    lcd.backlight();
```

```
    // Initialize LED pin as OUTPUT
```

```
    pinMode(LED_PIN, OUTPUT);
```

```
    // Start serial communication for the monitor
```

```
    Serial.begin(9600);
```

```
    // Initial message on LCD
```

```
    lcd.setCursor(0, 0);
```

```
    lcd.print("LDR Sensor");
```

```
    delay(1000);
```

```
}
```

```
void loop() {
```

```
// Read the value from the LDR sensor
int ldrValue = analogRead(LDR_PIN);

// Display the LDR value on the serial monitor
Serial.print("LDR Value: ");
Serial.println(ldrValue);

// Map LDR value to range from 0 to 255
int ledBrightness = map(ldrValue, 0, 1023, 0, 255);

// Control LED brightness based on LDR value
analogWrite(LED_PIN, ledBrightness);

// Display LDR value and LED brightness on LCD
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("LDR Value: ");
lcd.print(ldrValue);

lcd.setCursor(0, 1);
lcd.print("LED Brightness: ");
lcd.print(ledBrightness);

delay(500); // Delay for readability
}
```

---

### Working Explanation:

#### 1. LCD Initialization:

- The **LiquidCrystal\_I2C** library is used to control the LCD. The LCD is initialized with address 0x27 (common I2C address), with 16 columns and 2 rows.

#### 2. LDR Sensor:

- The LDR sensor is connected to **Analog Pin A0**. When the ambient light changes, the resistance of the LDR changes, which alters the voltage across it, which we read using `analogRead()`.

### 3. LED Control:

- The **LED** is connected to **Digital Pin 9**. The brightness of the LED is controlled using **PWM (Pulse Width Modulation)**, which is done using the `analogWrite()` function.
- The brightness is proportional to the LDR reading. The **LDR value** ranges from 0 (dark) to 1023 (bright). We map this to a range of 0 to 255 (PWM range) to control the LED brightness.

### 4. Serial Monitor:

- The **Serial Monitor** displays the raw LDR value to show the ambient light intensity.
- The **LED brightness** is also displayed in the **Serial Monitor** based on the mapped value.

### 5. LCD Display:

- The LCD updates the LDR value and LED brightness in real-time, providing a visual representation of the light level and the LED's brightness.

---

### Additional Notes:

- **LDR Behavior:** The LDR will have a higher resistance when it's dark and a lower resistance when it's exposed to light. This changes the voltage divider's output, which the Arduino reads as an analog value.
- **LED Brightness Adjustment:** As the light level changes, the LED's brightness will change in real-time. If the environment is dark, the LED will glow dimly; if it's bright, the LED will glow brightly.

---

Would you like to proceed with **Q19** next?

**Q19: Use Ultrasonic Sensor and 3 LEDs. Design the Circuit to Glow the LEDs at Three Different Distances with Its Display on Serial Monitor.**

---

**Required Components:**

- **Arduino Uno**
  - **Ultrasonic Sensor (HC-SR04)**
  - **3 LEDs (any color)**
  - **220Ω Resistors** (for current limiting)
  - **Jumper Wires**
  - **Breadboard**
  - **16x2 LCD Display with I2C Interface** (optional for display)
- 

**Circuit Connections:**

**1. Ultrasonic Sensor (HC-SR04):**

- **VCC** pin to **5V** on Arduino.
- **GND** pin to **Ground (GND)** on Arduino.
- **TRIG** pin to **Digital Pin 9** on Arduino.
- **ECHO** pin to **Digital Pin 10** on Arduino.

**2. LEDs (3 LEDs):**

- Connect the **Anode** (long leg) of the first LED to **Digital Pin 3**.
- Connect the **Cathode** (short leg) of the first LED to **Ground (GND)** through a **220Ω resistor**.
- Connect the **Anode** of the second LED to **Digital Pin 4**.
- Connect the **Cathode** of the second LED to **Ground (GND)** through a **220Ω resistor**.
- Connect the **Anode** of the third LED to **Digital Pin 5**.
- Connect the **Cathode** of the third LED to **Ground (GND)** through a **220Ω resistor**.

**3. LCD Display (Optional):**

- **VCC** of the LCD → **5V** on Arduino.
- **GND** of the LCD → **Ground (GND)** on Arduino.
- **SDA** of the LCD → **A4** on Arduino (Data Line).

- **SCL** of the LCD → **A5** on Arduino (Clock Line).

---

**Full Working Code:**

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
// LCD initialization
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD address 0x27, 16 columns, 2 rows
```

```
// LED Pins
```

```
#define LED1 3 // Red LED
```

```
#define LED2 4 // Yellow LED
```

```
#define LED3 5 // Green LED
```

```
// Ultrasonic Sensor Pins
```

```
#define TRIG_PIN 9
```

```
#define ECHO_PIN 10
```

```
void setup() {
```

```
    // Initialize the LCD
```

```
    lcd.begin(16, 2);
```

```
    lcd.backlight();
```

```
    // Initialize LED pins as OUTPUT
```

```
    pinMode(LED1, OUTPUT);
```

```
    pinMode(LED2, OUTPUT);
```

```
    pinMode(LED3, OUTPUT);
```

```
    // Initialize ultrasonic sensor pins
```

```
pinMode(TRIG_PIN, OUTPUT);
pinMode(ECHO_PIN, INPUT);

// Start serial communication for the monitor
Serial.begin(9600);
}

long getDistance() {
    // Send a pulse to the TRIG pin to start the measurement
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    // Measure the time taken for the pulse to return on the ECHO pin
    long duration = pulseIn(ECHO_PIN, HIGH);

    // Calculate the distance in centimeters
    long distance = duration * 0.034 / 2;
    return distance;
}

void loop() {
    // Get the distance from the ultrasonic sensor
    long distance = getDistance();

    // Display the distance on the Serial Monitor
    Serial.print("Distance: ");
    Serial.print(distance);
    Serial.println(" cm");
}
```

```

// Glow LEDs based on distance
if (distance < 10) {
    // If distance is less than 10 cm, glow Red LED
    digitalWrite(LED1, HIGH); // Red LED
    digitalWrite(LED2, LOW);
    digitalWrite(LED3, LOW);
} else if (distance >= 10 && distance < 20) {
    // If distance is between 10 cm and 20 cm, glow Yellow LED
    digitalWrite(LED1, LOW);
    digitalWrite(LED2, HIGH); // Yellow LED
    digitalWrite(LED3, LOW);
} else {
    // If distance is greater than or equal to 20 cm, glow Green LED
    digitalWrite(LED1, LOW);
    digitalWrite(LED2, LOW);
    digitalWrite(LED3, HIGH); // Green LED
}

// Display distance on LCD
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Distance: ");
lcd.print(distance);
lcd.print(" cm");

// Wait for 500ms before next measurement
delay(500);
}

```

---

**Working Explanation:**



### 1. LCD Initialization:

- The **LiquidCrystal\_I2C** library is used to control the LCD. The LCD is initialized with address 0x27, with 16 columns and 2 rows.

### 2. Ultrasonic Sensor:

- The **Ultrasonic sensor (HC-SR04)** is used to measure distance. The **TRIG** pin sends a pulse to the sensor, which then returns the pulse to the **ECHO** pin after bouncing off an object.
- The Arduino calculates the time it takes for the pulse to travel, and using the formula  $\text{distance} = \text{duration} * 0.034 / 2$ , it calculates the distance in centimeters.

### 3. LEDs:

- **LED1 (Red)** glows when the distance is less than 10 cm.
- **LED2 (Yellow)** glows when the distance is between 10 cm and 20 cm.
- **LED3 (Green)** glows when the distance is greater than or equal to 20 cm.

### 4. Serial Monitor:

- The **Serial Monitor** displays the current distance measured by the ultrasonic sensor in real-time.

### 5. LCD Display:

- The LCD continuously updates to show the current distance value, providing a real-time visual of the measurements.

---

### Additional Notes:

- You can adjust the distance thresholds for the LEDs based on your preference or application needs.
  - The LCD display provides a clear readout of the measured distance, while the LEDs visually indicate whether the object is within certain proximity ranges.
- 

Would you like to continue with **Q20** next?

**Q20: Design the Circuit for a Servo Motor with Potentiometer. It Is Expected that the Servo Motor Moves with Change in Potentiometer Position. Display the Digital Values in the Serial Monitor.**

---

**Required Components:**

- **Arduino Uno**
  - **Servo Motor**
  - **Potentiometer (10k $\Omega$ )**
  - **Jumper Wires**
  - **Breadboard**
  - **16x2 LCD Display with I2C Interface** (optional for display)
- 

**Circuit Connections:**

**1. Servo Motor:**

- Connect the **VCC** of the servo to **5V** on Arduino.
- Connect the **GND** of the servo to **Ground (GND)** on Arduino.
- Connect the **Control (Signal)** pin of the servo to **Digital Pin 9** on Arduino.

**2. Potentiometer:**

- Connect the **one side terminal (left)** of the potentiometer to **5V** on Arduino.
- Connect the **other side terminal (right)** of the potentiometer to **Ground (GND)** on Arduino.
- Connect the **wiper terminal (middle pin)** of the potentiometer to **Analog Pin A0** on Arduino.

**3. LCD Display (Optional):**

- **VCC** of the LCD → **5V** on Arduino.
  - **GND** of the LCD → **Ground (GND)** on Arduino.
  - **SDA** of the LCD → **A4** on Arduino (Data Line).
  - **SCL** of the LCD → **A5** on Arduino (Clock Line).
- 

**Full Working Code:**

cpp

CopyEdit

```
#include <Servo.h>

#include <Wire.h>

#include <LiquidCrystal_I2C.h>


// Initialize the Servo

Servo myServo;


// LCD initialization

LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD address 0x27, 16 columns, 2 rows


// Potentiometer pin

#define POT_PIN A0


void setup() {

    // Start the serial monitor for debugging

    Serial.begin(9600);


    // Attach the servo motor to pin 9

    myServo.attach(9);


    // Initialize LCD

    lcd.begin(16, 2);

    lcd.backlight();


    // Display a welcome message on LCD

    lcd.setCursor(0, 0);

    lcd.print("Servo Position:");

}


void loop() {

    // Read the potentiometer value (0 to 1023)
```

```
int potValue = analogRead(POT_PIN);

// Map the potentiometer value to a range of 0 to 180 (servo angle)
int angle = map(potValue, 0, 1023, 0, 180);

// Move the servo to the new angle
myServo.write(angle);

// Display the potentiometer value and the servo angle on the serial monitor
Serial.print("Potentiometer Value: ");
Serial.print(potValue);
Serial.print("\t Servo Angle: ");
Serial.println(angle);

// Display the servo angle on the LCD
lcd.setCursor(0, 1);
lcd.print("Angle: ");
lcd.print(angle);

// Wait a short time for smooth updates
delay(15);
}
```

---

### Working Explanation:

#### 1. LCD Initialization:

- The **LiquidCrystal\_I2C** library is used to control the LCD. The LCD is initialized with address 0x27 (common for most I2C LCDs), with 16 columns and 2 rows.
- The LCD displays the servo angle in real-time.

#### 2. Potentiometer:

- The **Potentiometer** is connected to the **Analog Pin A0**. It provides a variable resistance that the Arduino can read as an analog value ranging from 0 to 1023.

- The value from the potentiometer is then mapped to a range of 0 to 180, which corresponds to the servo's angle.

### 3. **Servo Motor:**

- The **Servo Motor** is controlled by the **Servo library**, and the angle is adjusted based on the potentiometer value.
- The **Servo** is attached to **Digital Pin 9** on the Arduino.

### 4. **Serial Monitor:**

- The **Serial Monitor** displays both the potentiometer value and the corresponding servo angle, providing real-time feedback.

### 5. **LCD Display:**

- The LCD continuously updates to show the current angle of the servo, allowing a visual display of the servo's movement.

---

#### **Additional Notes:**

- The **map()** function is used to convert the potentiometer's value (ranging from 0 to 1023) into an angle value (ranging from 0 to 180).
- The servo's angle will change in real-time as you adjust the potentiometer.
- The LCD will continuously update the servo angle, providing a visual output.

---

Would you like to continue with **Q21** next?

**Q21: Design the Circuit for an Infrared Sensor. Use Two or Three LEDs to Glow at Different Values. Display the Value on the Serial Monitor.**

---

**Required Components:**

- **Arduino Uno**
  - **Infrared (IR) Sensor**
  - **Red LED** (to indicate "no obstruction")
  - **Green LED** (to indicate "some obstruction")
  - **Yellow LED** (to indicate "maximum obstruction")
  - **Jumper Wires**
  - **Breadboard**
  - **16x2 LCD Display with I2C Interface** (optional for display)
- 

**Circuit Connections:**

**1. Infrared Sensor:**

- Connect the **VCC** pin of the IR sensor to the **5V** pin on Arduino.
- Connect the **GND** pin of the IR sensor to the **Ground (GND)** on Arduino.
- Connect the **OUT** pin (signal output) of the IR sensor to **Digital Pin 2** on Arduino.

**2. LEDs:**

- **Red LED:**
  - Anode (long pin) → **Digital Pin 3** on Arduino.
  - Cathode (short pin) → **Ground (GND)** (through a current-limiting resistor, typically 220Ω).
- **Green LED:**
  - Anode (long pin) → **Digital Pin 4** on Arduino.
  - Cathode (short pin) → **Ground (GND)** (through a current-limiting resistor, typically 220Ω).
- **Yellow LED:**
  - Anode (long pin) → **Digital Pin 5** on Arduino.
  - Cathode (short pin) → **Ground (GND)** (through a current-limiting resistor, typically 220Ω).

**3. LCD Display (Optional):**

- **VCC** of the LCD → **5V** on Arduino.
  - **GND** of the LCD → **Ground (GND)** on Arduino.
  - **SDA** of the LCD → **A4** on Arduino (Data Line).
  - **SCL** of the LCD → **A5** on Arduino (Clock Line).
- 

### Full Working Code:

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#define IR_SENSOR_PIN 2 // IR sensor signal input
```

```
#define RED_LED_PIN 3 // Red LED (no obstruction)
```

```
#define GREEN_LED_PIN 4 // Green LED (some obstruction)
```

```
#define YELLOW_LED_PIN 5 // Yellow LED (maximum obstruction)
```

```
// Initialize LCD
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD address 0x27, 16 columns, 2 rows
```

```
void setup() {
```

```
    // Start serial communication
```

```
    Serial.begin(9600);
```

```
    // Set pin modes
```

```
    pinMode(IR_SENSOR_PIN, INPUT);
```

```
    pinMode(RED_LED_PIN, OUTPUT);
```

```
    pinMode(GREEN_LED_PIN, OUTPUT);
```

```
    pinMode(YELLOW_LED_PIN, OUTPUT);
```

```
    // Initialize LCD
```

```
    lcd.begin(16, 2);
```

```
lcd.backlight();

// Display a welcome message on LCD
lcd.setCursor(0, 0);
lcd.print("IR Sensor Demo");
}

void loop() {
    // Read the value from the infrared sensor
    int irValue = digitalRead(IR_SENSOR_PIN);

    // Display the IR sensor value on the serial monitor
    Serial.print("IR Sensor Value: ");
    Serial.println(irValue);

    // Turn LEDs on or off based on IR sensor value
    if (irValue == LOW) {
        // No obstruction
        digitalWrite(RED_LED_PIN, HIGH);
        digitalWrite(GREEN_LED_PIN, LOW);
        digitalWrite(YELLOW_LED_PIN, LOW);
        lcd.setCursor(0, 1);
        lcd.print("No Obstruction ");
    }
    else if (irValue == HIGH) {
        // Obstruction detected
        digitalWrite(RED_LED_PIN, LOW);
        digitalWrite(GREEN_LED_PIN, HIGH);
        digitalWrite(YELLOW_LED_PIN, LOW);
        lcd.setCursor(0, 1);
        lcd.print("Obstruction Detected ");
    }
}
```



```
}
```

```
// Short delay for readability
```

```
delay(500);
```

```
}
```

---

### Working Explanation:

#### 1. Infrared Sensor:

- The IR sensor is used to detect objects or obstacles. The output pin of the IR sensor gives a **LOW** signal (0) when there is no obstruction and a **HIGH** signal (1) when an object is detected.

#### 2. LED Indicators:

- **Red LED:** Lights up when there is **no obstruction**.
- **Green LED:** Lights up when there is an **obstruction** detected.
- **Yellow LED:** (Not used in this specific setup but can be used for different thresholds if needed).

#### 3. LCD Display (Optional):

- The **LCD Display** shows the status of the IR sensor:
  - "No Obstruction" when no object is detected.
  - "Obstruction Detected" when an object is detected.
- This helps in visualizing the status of the IR sensor, in addition to the serial monitor output.

#### 4. Serial Monitor:

- The **Serial Monitor** shows the raw value of the IR sensor, indicating whether the sensor is detecting an obstruction or not.

---

### Additional Notes:

- The **IR sensor** used here typically operates with a logic-level output, where the signal is **LOW** when no obstruction is detected and **HIGH** when there is an obstruction.
- The **LCD** helps provide a visual display of the system's state in addition to the **Serial Monitor**, which is used for debugging.

---

Would you like to proceed with **Q22** next?

**Q22: Design the Circuit for a DC Motor with a Relay. It is Expected to Glow Two LEDs to Represent ON/OFF Status of the Motor.**

---

**Required Components:**

- **Arduino Uno**
  - **DC Motor**
  - **Relay Module (5V)**
  - **Red LED** (for OFF status)
  - **Green LED** (for ON status)
  - **Diode** (1N4007) – for back EMF protection (optional but recommended)
  - **Flyback Diode** (for motor protection)
  - **Jumper Wires**
  - **Breadboard**
  - **Power Source** (for the motor, typically 12V)
- 

**Circuit Connections:**

**1. DC Motor & Relay:**

- **Relay Module:**
  - **VCC** of the relay → **5V** on Arduino.
  - **GND** of the relay → **GND** on Arduino.
  - **IN** (control signal) of the relay → **Digital Pin 8** on Arduino.
  - **NO (Normally Open)** pin of the relay → one terminal of the **DC Motor**.
  - **COM (Common)** pin of the relay → **Positive terminal** of the DC motor power supply (12V).
  - **DC Motor:** The **negative terminal** of the motor goes to the **GND** of the power supply.

**2. LEDs:**

- **Red LED** (OFF Status):
  - Anode (long pin) → **Digital Pin 3** on Arduino.
  - Cathode (short pin) → **Ground (GND)** through a **220Ω** current-limiting resistor.
- **Green LED** (ON Status):

- Anode (long pin) → **Digital Pin 4** on Arduino.
- Cathode (short pin) → **Ground (GND)** through a **220Ω** current-limiting resistor.

### 3. Power Supply for the Motor:

- If the motor operates at a different voltage (12V or more), connect the **positive terminal of the motor power supply** to the **COM pin of the relay**.
- The **negative terminal** of the motor power supply goes to **GND**.
- The **Arduino** can be powered via USB or its 5V pin.

---

### Full Working Code:

cpp

CopyEdit

```
#define RELAY_PIN 8    // Relay control pin
#define RED_LED_PIN 3  // Red LED for OFF status
#define GREEN_LED_PIN 4 // Green LED for ON status

void setup() {
    // Initialize pin modes
    pinMode(RELAY_PIN, OUTPUT);
    pinMode(RED_LED_PIN, OUTPUT);
    pinMode(GREEN_LED_PIN, OUTPUT);

    // Start serial communication
    Serial.begin(9600);

    // Initialize relay and LEDs to OFF
    digitalWrite(RELAY_PIN, LOW); // Motor OFF
    digitalWrite(RED_LED_PIN, HIGH); // Red LED ON (OFF status)
    digitalWrite(GREEN_LED_PIN, LOW); // Green LED OFF
}

void loop() {
```

```
// Simulating turning the motor ON

digitalWrite(RELAY_PIN, HIGH); // Turn the relay ON (motor ON)

digitalWrite(RED_LED_PIN, LOW); // Turn off Red LED

digitalWrite(GREEN_LED_PIN, HIGH); // Turn on Green LED

Serial.println("Motor ON");

delay(5000); // Motor runs for 5 seconds


// Simulating turning the motor OFF

digitalWrite(RELAY_PIN, LOW); // Turn the relay OFF (motor OFF)

digitalWrite(RED_LED_PIN, HIGH); // Turn on Red LED

digitalWrite(GREEN_LED_PIN, LOW); // Turn off Green LED

Serial.println("Motor OFF");

delay(5000); // Motor stays off for 5 seconds

}
```

---

### Working Explanation:

#### 1. Relay Module:

- The relay is used to control the power flow to the **DC motor**.
- When the **RELAY\_PIN** is set to **HIGH**, the relay switches the motor's power **ON**.
- When the **RELAY\_PIN** is set to **LOW**, the relay switches the motor's power **OFF**.

#### 2. LED Indicators:

- **Red LED:** Indicates when the **motor is OFF** (i.e., relay is in the **LOW** state).
- **Green LED:** Indicates when the **motor is ON** (i.e., relay is in the **HIGH** state).

#### 3. Power Supply for the Motor:

- The **DC motor** requires a separate power supply (12V in this case).
- The relay module acts as a switch, controlling whether the power from the motor's power supply reaches the motor.

#### 4. Serial Monitor:

- The **Serial Monitor** displays whether the motor is currently **ON** or **OFF** for debugging and monitoring purposes.
  - It prints the status ("Motor ON" or "Motor OFF") based on the relay's state.
-

**Additional Notes:**

- **Flyback Diode:** It is recommended to use a **flyback diode** (like 1N4007) across the motor's terminals to protect the relay from voltage spikes when turning the motor off. Connect the anode to the **GND** side and the cathode to the **positive terminal** of the motor.
- **Relay Module:** If you're using a **5V relay module**, it will work perfectly with Arduino's **5V** supply. Ensure the relay can handle the voltage and current requirements of your motor.

---

Would you like to proceed with **Q23** next?

## Q23: Design the Circuit Using a Potentiometer and LCD Display to Scroll 'YOUR NAME' on the LCD Screen

---

### Required Components:

- **Arduino Uno**
  - **Potentiometer** (10k $\Omega$ )
  - **16x2 LCD Display** (with I2C interface)
  - **Jumper Wires**
  - **Breadboard**
- 

### Circuit Connections:

#### 1. Potentiometer:

- **VCC** of the potentiometer → **5V** on Arduino.
- **GND** of the potentiometer → **GND** on Arduino.
- **Wiper (middle pin)** → **Analog Pin A0** on Arduino.

#### 2. LCD Display:

- **GND** of the LCD → **GND** on Arduino.
  - **VCC** of the LCD → **5V** on Arduino.
  - **SDA** of the LCD → **A4** on Arduino (I2C communication).
  - **SCL** of the LCD → **A5** on Arduino (I2C communication).
- 

### Full Working Code:

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#define POT_PIN A0 // Potentiometer connected to Analog pin A0
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // Initialize LCD with I2C address 0x27
```

```

void setup() {
  // Initialize LCD
  lcd.begin();
  lcd.backlight();

  // Display initial message
  lcd.setCursor(0, 0);
  lcd.print("Scroll Text:");
}

void loop() {
  // Read potentiometer value (range 0 to 1023)
  int potValue = analogRead(POT_PIN);

  // Map the potentiometer value to the scroll delay (shorter delay for higher values)
  int delayTime = map(potValue, 0, 1023, 100, 500);

  // Scroll "YOUR NAME" across the LCD
  lcd.setCursor(0, 1);
  lcd.print("YOUR NAME   "); // Ensure 16 characters (fill the line)
  lcd.scrollDisplayLeft();

  // Delay based on potentiometer value
  delay(delayTime);
}

```

---

### Working Explanation:

#### 1. Potentiometer:

- The potentiometer is connected to **Analog Pin A0** of the Arduino. It controls the scroll speed of the text on the LCD screen.
- The potentiometer's **wiper (middle pin)** reads the analog value, which is mapped to a range that controls the delay between scrolling text.

## 2. LCD Display:

- The **LiquidCrystal\_I2C** library is used to interface with the LCD through **I2C communication**.
- The **lcd.setCursor(0, 0)** places the text on the first line of the LCD, and **lcd.setCursor(0, 1)** places the text on the second line.
- The text "YOUR NAME" scrolls from right to left using **lcd.scrollDisplayLeft()**.
- The speed of the scrolling text is controlled by the potentiometer. The delay time between each scroll is mapped from the potentiometer's value, which ranges from 100ms (fast) to 500ms (slow).

## 3. Mapping Potentiometer to Delay:

- The **analogRead(POT\_PIN)** reads the potentiometer value (ranging from 0 to 1023).
- This value is then **mapped** to a delay time using **map()**, with the minimum value of the potentiometer resulting in a faster scroll and the maximum value resulting in a slower scroll.

## 4. Display on LCD:

- The LCD will continuously display and scroll "YOUR NAME" based on the potentiometer's position, with the speed adjusted dynamically.

---

### Additional Notes:

- Ensure the **potentiometer's** middle pin is connected to the **Analog Pin A0** for reading its value.
- The LCD's I2C address is commonly **0x27**; however, if your LCD uses a different address, update it in the `LiquidCrystal_I2C lcd(0x27, 16, 2)` line accordingly.
- You can modify the text being scrolled by changing "YOUR NAME" to whatever text you'd like.

---

Would you like to proceed with **Q24** next?



## Q24: Design the Circuit Using an LED to Make a Counter till 20 and Display on the LED Screen

---

### Required Components:

- **Arduino Uno**
  - **LED** (for display)
  - **220Ω Resistor** (for current limiting)
  - **16x2 LCD Display** (with I2C interface)
  - **Jumper Wires**
  - **Breadboard**
- 

### Circuit Connections:

#### 1. LED:

- **Anode (long leg)** → **Digital Pin 13** on Arduino.
- **Cathode (short leg)** → **GND** on Arduino.
- Connect a **220Ω resistor** between the LED's **anode** and **Arduino pin 13** to limit the current.

#### 2. LCD Display:

- **GND** of the LCD → **GND** on Arduino.
  - **VCC** of the LCD → **5V** on Arduino.
  - **SDA** of the LCD → **A4** on Arduino (I2C communication).
  - **SCL** of the LCD → **A5** on Arduino (I2C communication).
- 

### Full Working Code:

cpp

CopyEdit

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#define LED_PIN 13 // LED connected to Digital Pin 13
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // Initialize LCD with I2C address 0x27
```

```
int counter = 0; // Counter variable
```

```
void setup() {
```

```
    // Initialize LED pin
```

```
    pinMode(LED_PIN, OUTPUT);
```

```
    // Initialize LCD
```

```
    lcd.begin();
```

```
    lcd.backlight();
```

```
    // Display initial message on LCD
```

```
    lcd.setCursor(0, 0);
```

```
    lcd.print("Counting till 20");
```

```
}
```

```
void loop() {
```

```
    // Turn on the LED for each count from 1 to 20
```

```
    if (counter < 20) {
```

```
        digitalWrite(LED_PIN, HIGH); // Turn on LED
```

```
        lcd.setCursor(0, 1); // Move to the second line on LCD
```

```
        lcd.print("Count: ");
```

```
        lcd.print(counter); // Display the counter value on LCD
```

```
        counter++; // Increment counter
```

```
        delay(1000); // Wait for 1 second
```

```
        digitalWrite(LED_PIN, LOW); // Turn off LED
```

```
    } else {
```

```
        // Once the counter reaches 20, reset it and stop the process
```

```
        counter = 0;
```

```
        lcd.clear();
```

```
        lcd.setCursor(0, 0);
```

```
lcd.print("Counting reset!");  
  
delay(2000); // Display reset message for 2 seconds  
  
lcd.clear();  
  
lcd.setCursor(0, 0);  
  
lcd.print("Counting till 20");  
  
}  
  
}
```

---

### Working Explanation:

#### 1. LED:

- The **LED** connected to **Digital Pin 13** will blink every time the counter increments.
- **digitalWrite(LED\_PIN, HIGH)** turns the LED on, and **digitalWrite(LED\_PIN, LOW)** turns it off.
- The LED stays on for 1 second during each count, then turns off while the counter is incremented.

#### 2. LCD Display:

- The LCD will show the message "Counting till 20" on the first line when the program starts.
- The counter value will be displayed on the second line of the LCD.
- After reaching 20, the LCD will show a message "Counting reset!" for 2 seconds, after which it resets the counter to 0 and starts counting again.

#### 3. Counter Logic:

- The variable counter starts at 0 and increments by 1 with each loop iteration.
- The counter is displayed on the LCD, and the LED is turned on during each increment. After counting to 20, the process restarts.

#### 4. Reset and Delay:

- Once the counter reaches 20, the LCD displays a reset message for 2 seconds before clearing the screen and restarting the count.
  - The process continues indefinitely.
- 

### Additional Notes:

- Ensure the **LED** is connected properly to **Digital Pin 13**.

- The **220Ω resistor** limits the current through the LED to protect it from excessive current.
  - The LCD's **I2C address** is set to **0x27**, but if your LCD has a different address, you can adjust it in the `LiquidCrystal_I2C lcd(0x27, 16, 2)` line.
  - You can modify the counter behavior or the display message as needed by editing the code.
- 

Would you like to proceed with **Q25** next?

**Q25: Design the Circuit for DC Motor with Relay. It is Expected to Glow the Two LEDs to Represent ON/OFF Status of Motor.**

---

**Required Components:**

- **Arduino Uno**
  - **DC Motor**
  - **Relay Module**
  - **2 LEDs (for ON/OFF status)**
  - **Diode (1N4007)** (for flyback protection)
  - **220Ω Resistors** (for current limiting the LEDs)
  - **Jumper Wires**
  - **Breadboard**
- 

**Circuit Connections:**

**1. DC Motor and Relay:**

- Connect one terminal of the **DC motor** to the **Normally Open (NO)** terminal of the relay.
- Connect the other terminal of the **DC motor** to **GND**.
- The **Common (COM)** terminal of the relay connects to **5V**.
- Place a **flyback diode** (1N4007) across the DC motor terminals. Connect the cathode (marked end) to **5V** and the anode to the motor terminal connected to the relay's **NO** terminal.

**2. LEDs:**

- **LED1 (Green)** for "Motor ON":
  - Anode → **Digital Pin 3**.
  - Cathode → **GND** (through a 220Ω resistor).
- **LED2 (Red)** for "Motor OFF":
  - Anode → **Digital Pin 4**.
  - Cathode → **GND** (through a 220Ω resistor).

**3. Relay Module:**

- **VCC** of the relay module → **5V** on Arduino.
- **GND** of the relay module → **GND** on Arduino.

- **IN** pin of the relay module → **Digital Pin 7** on Arduino (to control relay).

---

### Full Working Code:

cpp

CopyEdit

```
#define LED1 3 // Green LED for Motor ON
```

```
#define LED2 4 // Red LED for Motor OFF
```

```
#define RELAY_PIN 7 // Relay control pin
```

```
void setup() {
```

```
    // Initialize pins
```

```
    pinMode(LED1, OUTPUT);
```

```
    pinMode(LED2, OUTPUT);
```

```
    pinMode(RELAY_PIN, OUTPUT);
```

```
    // Turn off both LEDs initially
```

```
    digitalWrite(LED1, LOW);
```

```
    digitalWrite(LED2, HIGH);
```

```
    // Set relay to OFF
```

```
    digitalWrite(RELAY_PIN, LOW);
```

```
}
```

```
void loop() {
```

```
    // Turn the motor ON
```

```
    digitalWrite(RELAY_PIN, HIGH); // Activate relay to power the motor
```

```
    digitalWrite(LED1, HIGH); // Green LED ON (Motor ON)
```

```
    digitalWrite(LED2, LOW); // Red LED OFF (Motor OFF)
```

```
    delay(5000); // Motor ON for 5 seconds
```

```
// Turn the motor OFF

digitalWrite(RELAY_PIN, LOW); // Deactivate relay to turn off the motor

digitalWrite(LED1, LOW); // Green LED OFF (Motor OFF)

digitalWrite(LED2, HIGH); // Red LED ON (Motor OFF)


delay(5000); // Motor OFF for 5 seconds

}
```

---

### Working Explanation:

#### 1. DC Motor and Relay:

- The **relay** module acts as a switch to control the DC motor.
- When the relay is activated (using **HIGH** on the **IN** pin), it closes the NO (Normally Open) contacts, allowing current to flow to the motor, thereby turning it ON.
- When the relay is deactivated (using **LOW** on the **IN** pin), it opens the contacts, cutting power to the motor, thus turning it OFF.

#### 2. LEDs for Status:

- **LED1 (Green)** is used to indicate when the motor is ON.
- **LED2 (Red)** is used to indicate when the motor is OFF.
- The status of the LEDs is controlled in the **loop** based on the relay's state.

#### 3. Code Logic:

- Initially, both LEDs are set to show the motor is OFF (Red LED ON, Green LED OFF).
- The motor is powered ON for 5 seconds (Relay activated, Green LED ON).
- Then the motor is powered OFF for 5 seconds (Relay deactivated, Red LED ON).
- This loop repeats indefinitely.

#### 4. Flyback Diode:

- The **flyback diode (1N4007)** is used across the motor terminals to protect the relay and Arduino from voltage spikes when the motor turns OFF (inductive load).

---

### Additional Notes:

- Make sure the **relay module** is rated for the voltage and current of your DC motor.

- Use a **flyback diode** to prevent damage from inductive kickback when switching the motor.
  - If you're using a motor that requires higher current than the Arduino can provide, a relay module with an appropriate driver transistor is essential.
  - Adjust the **delay** in the loop if you want the motor to stay ON/OFF for different durations.
- 

Would you like to proceed with **Q26**?



**Q26: Design the Circuit for Servo Motor with Potentiometer. It is Expected That Servo Motor Moves with Change in Potentiometer Position. Display the Digital Values in the Serial Monitor.**

---

**Required Components:**

- **Arduino Uno**
  - **Servo Motor**
  - **Potentiometer (10k $\Omega$ )**
  - **Jumper Wires**
  - **Breadboard**
- 

**Circuit Connections:**

**1. Servo Motor:**

- Connect the **VCC pin** of the **servo motor** to **5V** on the Arduino.
- Connect the **GND pin** of the **servo motor** to **GND** on the Arduino.
- Connect the **Signal pin** of the **servo motor** to **Digital Pin 9** on the Arduino.

**2. Potentiometer:**

- Connect the **VCC pin** of the **potentiometer** to **5V** on the Arduino.
  - Connect the **GND pin** of the **potentiometer** to **GND** on the Arduino.
  - Connect the **Wiper (middle pin)** of the **potentiometer** to **Analog Pin A0** on the Arduino (this will give the variable voltage input).
- 

**Full Working Code:**

cpp

CopyEdit

```
#include <Servo.h>
```

```
#define POT_PIN A0 // Potentiometer connected to analog pin A0
```

```
#define SERVO_PIN 9 // Servo motor connected to digital pin 9
```

```
Servo myServo; // Create a Servo object
```

```
int potValue = 0; // Variable to store potentiometer value (0-1023)
```

```
int angle = 0; // Variable to store the calculated angle (0-180)

void setup() {
  myServo.attach(SERVO_PIN); // Attach the servo motor to pin 9
  Serial.begin(9600); // Initialize serial communication at 9600 baud
}

void loop() {
  potValue = analogRead(POT_PIN); // Read the potentiometer value (0 to 1023)

  angle = map(potValue, 0, 1023, 0, 180); // Map the potentiometer value to a range of 0-180 (for servo angle)

  myServo.write(angle); // Set the servo motor position based on the mapped value
  Serial.print("Potentiometer Value: ");
  Serial.print(potValue); // Print the potentiometer value to the Serial Monitor
  Serial.print(" Servo Angle: ");
  Serial.println(angle); // Print the calculated servo angle

  delay(15); // Wait for the servo to reach the position
}
```

---

### Working Explanation:

#### 1. Servo Motor:

- The **servo motor** is controlled by sending a PWM signal to its **signal pin** (pin 9 in this case). The servo's position is adjusted according to the input signal, which in this case is determined by the potentiometer.

#### 2. Potentiometer:

- The **potentiometer** acts as an analog input to the Arduino. By turning the potentiometer, the **wiper** provides a voltage between **0V and 5V**, which is read by the **analog input pin** (A0).
- The value read by the potentiometer will range from **0** (for 0V) to **1023** (for 5V).

#### 3. Mapping:

- The **analogRead** function reads the potentiometer's value in the range of **0 to 1023**. This value is then mapped to a range of **0 to 180** using the **map()** function, which corresponds to the servo's angle range.

#### 4. **Servo Control:**

- The **servo motor** is moved to the calculated angle using the **myServo.write(angle)** command. This rotates the servo to the angle corresponding to the potentiometer's position.

#### 5. **Serial Monitor:**

- The potentiometer value and the corresponding servo angle are displayed on the **Serial Monitor** for debugging and monitoring.
- This helps in seeing the digital values read from the potentiometer and how they affect the servo motor's movement.

#### 6. **Delay:**

- A short **delay(15)** is added to ensure the servo has enough time to move to the desired position.

---

#### **Additional Notes:**

- The **servo motor** may need a separate **power supply** if it requires more current than the Arduino can supply. In that case, ensure to connect the external power supply's **GND** to the **Arduino GND**.
- You can adjust the potentiometer to see how the servo motor's position changes in real time.
- If you need to adjust the servo's range (e.g., limiting the servo's movement to 45-135 degrees), you can modify the **map()** function accordingly.